# CHƯƠNG 3: THIẾT KẾ KIẾN TRÚC HỆ THỐNG

## 3.1 Tổng quan hệ thống

Hệ thống SoC được xây dựng theo kiến trúc bus tập trung, trong đó tất cả các thành phần giao tiếp với nhau thông qua một crossbar switch sử dụng giao thức TileLink Uncached Lightweight (TL-UL). Toàn bộ hệ thống hoạt động trong một miền xung nhịp duy nhất, giúp đơn giản hóa thiết kế và loại bỏ các vấn đề liên quan đến đồng bộ hóa giữa các miền xung nhịp khác nhau.

Crossbar được cấu hình với **2 master và 6 slave (2M6S)**:

- **Master 0 – CPU**: Lõi RISC-V pipeline 5 tầng. Phát sinh các giao dịch TileLink tại tầng MEM để thực hiện lệnh load/store.
- **Master 1 – DMA Controller**: Bộ điều khiển DMA 2 kênh. Chủ động phát sinh các giao dịch đọc/ghi bộ nhớ mà không cần sự can thiệp của CPU.

Sáu slave trong hệ thống được phân bổ địa chỉ như Bảng 3.1.

**Bảng 3.1 – Bảng phân bổ địa chỉ hệ thống**

| Slave | Tên module | Địa chỉ base | Kích thước | Mô tả |
|---|---|---|---|---|
| S0 | DMEM Adapter | 0x0000_0000 | 32 KB | Bộ nhớ dữ liệu và chương trình |
| S1 | Peripheral Adapter | 0x1000_0000 | 256 B | LED đỏ, LED xanh, HEX, GPIO |
| S2 | UART Bridge | 0x1002_0000 | 64 B | Giao tiếp nối tiếp 115200 baud |
| S3 | DMA Registers | 0x1005_0000 | 256 B | Thanh ghi cấu hình DMA |
| S4 | ChaCha20 | 0x1006_0000 | 256 B | Bộ tăng tốc mã hóa dòng |
| S5 | Poly1305 | 0x1007_0000 | 128 B | Bộ tăng tốc xác thực MAC |

Sơ đồ khối tổng thể của hệ thống được trình bày trong Hình 3.1. Mỗi kết nối giữa master/slave và crossbar gồm hai kênh: **Channel A** mang yêu cầu (request) từ master đến slave, **Channel B** mang phản hồi (response) theo chiều ngược lại. DMA Controller có vai trò kép: đóng vai trò **Master** khi chủ động truy cập bộ nhớ qua Master Interface, và đóng vai trò **Slave** khi CPU ghi vào các thanh ghi cấu hình qua Slave Interface.

*[Chèn Hình 3.1 – SoC Block Diagram]*

Tín hiệu reset bên ngoài `rst_n` được đồng bộ hóa bằng mạch hai flip-flop trước khi phân phối đến tất cả các module bên trong, đảm bảo tránh hiện tượng metastability khi reset được giải phóng không đồng bộ với xung nhịp hệ thống.

---

## 3.2 Kiến trúc CPU RISC-V

Lõi vi xử lý được xây dựng theo kiến trúc RISC-V RV32I với pipeline 5 tầng cổ điển: **IF → ID → EX → MEM → WB**. Phần này trình bày tóm tắt kiến trúc CPU từ công trình kỳ trước, tập trung vào những phần có liên quan trực tiếp đến việc mở rộng hệ thống trong đồ án này.

### 3.2.1 Cấu trúc pipeline

Pipeline gồm 5 tầng được kết nối qua 4 thanh ghi pipeline: IF/ID, ID/EX, EX/MEM và MEM/WB. Mỗi tầng thực hiện một công việc độc lập trong cùng một chu kỳ xung nhịp, cho phép tối đa 5 lệnh cùng tồn tại trong pipeline ở các giai đoạn khác nhau.

- **IF (Instruction Fetch)**: Đọc lệnh từ bộ nhớ theo địa chỉ PC hiện tại. PC được cập nhật tuần tự (+4) hoặc nhảy đến địa chỉ đích khi có lệnh rẽ nhánh.
- **ID (Instruction Decode)**: Giải mã lệnh, đọc giá trị thanh ghi nguồn từ register file, sinh immediate và các tín hiệu điều khiển.
- **EX (Execute)**: Thực hiện phép tính ALU, tính địa chỉ branch target và phát hiện misprediction.
- **MEM (Memory Access)**: Thực hiện truy cập bộ nhớ thông qua giao thức TileLink-UL. Đây là tầng được mở rộng đáng kể so với thiết kế gốc.
- **WB (Write Back)**: Ghi kết quả trở lại register file.

### 3.2.2 Xử lý hazard

Pipeline tích hợp hai cơ chế xử lý hazard:

**Forwarding Unit**: Phát hiện và giải quyết data hazard bằng cách chuyển tiếp kết quả từ tầng MEM hoặc WB trực tiếp về đầu vào tầng EX, tránh phải stall pipeline trong hầu hết các trường hợp. Forwarding unit hỗ trợ cả trường hợp JAL/JALR cần chuyển tiếp giá trị PC+4.

**Hazard Detection Unit**: Phát hiện các trường hợp không thể giải quyết bằng forwarding, bao gồm load-use hazard (lệnh load theo sau ngay bởi lệnh sử dụng kết quả) và misprediction. Khi phát hiện load-use hazard, đơn vị này chèn một NOP bubble vào tầng ID/EX và giữ nguyên các tầng IF và ID trong một chu kỳ.

### 3.2.3 Giao tiếp TileLink tại tầng MEM

Đây là điểm mở rộng quan trọng nhất của CPU trong bối cảnh tích hợp SoC. Thay vì truy cập bộ nhớ trực tiếp, tầng MEM (module `MEM_tl`) phát sinh các giao dịch TileLink-UL chuẩn:

- Lệnh **load**: Phát gói Get (opcode = 4) trên Channel A với địa chỉ tính toán từ ALU. Pipeline stall cho đến khi nhận được AccessAckData trên Channel D.
- Lệnh **store**: Phát gói PutFullData (opcode = 0) trên Channel A kèm dữ liệu cần ghi. Pipeline stall cho đến khi nhận được AccessAck trên Channel D.

Tín hiệu `lsu_stall` được kéo lên mức 1 trong suốt thời gian chờ phản hồi TileLink. Tín hiệu này tác động đến toàn bộ pipeline thông qua logic stall/flush:

```
pipeline_stall  = lsu_stall
combined_stall  = stall | pipeline_stall   -- giữ IF, ID, EX
combined_flush  = mispred & ~pipeline_stall -- flush bị trì hoãn khi TL đang
bận
```

Thiết kế này đảm bảo rằng nếu phát hiện misprediction trong khi pipeline đang stall do TileLink, lệnh flush sẽ được trì hoãn đến khi giao dịch TileLink hoàn thành, tránh mất dữ liệu hoặc flush không đúng tầng.

---

## 3.3 TileLink Crossbar 2M6S

Module `tlul_xbar_2m6s` đóng vai trò trung tâm kết nối toàn bộ hệ thống, cho phép 2 master giao tiếp với 6 slave thông qua một tập hợp các multiplexer và logic phân xử truy cập bus.

### 3.3.1 Giải mã địa chỉ

Mỗi yêu cầu từ master mang theo địa chỉ 32-bit. Crossbar kiểm tra các bit địa chỉ cao để xác định slave đích theo quy tắc:

| Điều kiện địa chỉ | Slave được chọn |
|---|---|
| `addr[31:15] == 0` | S0 – DMEM Adapter |
| `addr[27:24] == 4'h1` và `addr[23:20] == 4'h0` và `addr[19:16] == 4'h0` | S1 – Peripheral |
| `addr[23:16] == 8'h02` | S2 – UART |
| `addr[23:16] == 8'h05` | S3 – DMA Registers |
| `addr[23:16] == 8'h06` | S4 – ChaCha20 |
| `addr[23:16] == 8'h07` | S5 – Poly1305 |

Kết quả giải mã tạo ra tín hiệu `slave_sel[5:0]` chỉ định đúng một slave được kích hoạt cho mỗi giao dịch.

### 3.3.2 Cơ chế phân xử Round-Robin

Khi cả hai master đồng thời phát sinh yêu cầu đến cùng một slave, crossbar sử dụng thuật toán **Round-Robin** để phân xử công bằng. Trạng thái phân xử được quản lý bởi thanh ghi `rr_priority`: khi bằng 0 thì CPU được ưu tiên, khi bằng 1 thì DMA được ưu tiên. Sau mỗi lần cấp quyền truy cập, `rr_priority` được đảo chiều để đảm bảo hai master luân phiên nhau khi cùng tranh chấp.

Crossbar hoạt động theo mô hình **transaction-based**: một giao dịch được coi là hoàn thành khi nhận được tín hiệu phản hồi hợp lệ trên Channel D. Trong thời gian một giao dịch đang xử lý, master kia có thể được cấp quyền truy cập các slave khác nếu không xảy ra xung đột.

### 3.3.3 Đồng bộ hóa reset

Tín hiệu reset bên ngoài `rst_n` có thể thay đổi bất đồng bộ so với xung nhịp hệ thống, dẫn đến nguy cơ metastability tại đầu vào của các flip-flop trong hệ thống. Để khắc phục, một mạch đồng bộ hóa hai tầng (two-stage synchronizer) được sử dụng:

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        rst_n_sync_1 <= 1'b0;
        rst_n_sync   <= 1'b0;
    end else begin
        rst_n_sync_1 <= 1'b1;
        rst_n_sync   <= rst_n_sync_1;
    end
end
```

Tín hiệu `rst_n_sync` sau khi qua hai flip-flop được phân phối đến tất cả các module, đảm bảo reset được giải phóng đồng bộ với xung nhịp và không gây bất ổn định tín hiệu.

---

## 3.4 DMA Controller

*(Chuyển nội dung từ BaoCao.docx vào đây — đã có đầy đủ các mục: tổng quan, register map, FSM channel, arbiter, DMA master)*

---

## 3.5 ChaCha20 Hardware Accelerator

### 3.5.1 Tổng quan

ChaCha20 là thuật toán mã hóa dòng (stream cipher) được định nghĩa trong RFC 8439. Bộ tăng tốc phần cứng ChaCha20 trong hệ thống SoC này bao gồm ba lớp như Hình 3.X:

- **chacha20_qr**: Module tính toán hàm Quarter Round, hoàn toàn tổ hợp (combinatorial).
- **chacha20_core**: Lõi tính toán điều khiển bởi FSM, sử dụng 4 instance của chacha20_qr chạy song song.
- **tlul_chacha20**: Lớp giao tiếp TileLink-UL, cung cấp register map cho CPU và kết nối với chacha20_core.

*[Chèn Hình 3.X – Sơ đồ khối ChaCha20 Accelerator]*

### 3.5.2 Hàm Quarter Round (chacha20_qr)

Hàm Quarter Round là đơn vị tính toán cơ bản của ChaCha20. Mỗi lần gọi nhận 4 đầu vào 32-bit (a, b, c, d) và tạo ra 4 đầu ra 32-bit thông qua chuỗi 4 phép ARX (Add-Rotate-XOR):

```
a += b;  d ^= a;  d <<<= 16
c += d;  b ^= c;  b <<<= 12
a += b;  d ^= a;  d <<<= 8
c += d;  b ^= c;  b <<<= 7
```

Toàn bộ hàm này được hiện thực bằng logic tổ hợp thuần túy, không cần xung nhịp, với độ trễ bằng đúng một tầng logic cộng và XOR. Bốn phép xoay trái (rotate left) được hiện thực bằng cách đấu nối lại thứ tự bit mà không tốn bất kỳ tài nguyên logic nào:

```systemverilog
assign d1 = {dx1[15:0], dx1[31:16]};  // rotl 16
assign b1 = {bx1[19:0], bx1[31:20]};  // rotl 12
assign d2 = {dx2[23:0], dx2[31:24]};  // rotl 8
assign b2 = {bx2[24:0], bx2[31:25]};  // rotl 7
```

### 3.5.3 Lõi tính toán (chacha20_core)

Lõi tính toán quản lý toàn bộ quá trình tạo keystream 512-bit (16 word 32-bit) thông qua FSM 4 trạng thái.

**Ma trận khởi tạo**: Theo RFC 8439 §2.3, ma trận 4×4 được xây dựng từ:

- Word 0–3: Hằng số "expand 32-byte k" (sigma)
- Word 4–11: Khóa bí mật 256-bit
- Word 12: Block counter
- Word 13–15: Nonce 96-bit

**FSM điều khiển**:

*[Chèn Hình 3.X – State diagram FSM ChaCha20 Core]*

| Trạng thái | Mô tả | Số chu kỳ |
|---|---|---|
| S_IDLE | Chờ lệnh start, nạp ma trận khởi tạo | 1 |
| S_ROUND | Thực hiện 20 vòng (10 column + 10 diagonal) | 20 |
| S_ADD | Cộng ma trận làm việc với ma trận khởi tạo | 1 |
| S_VALID | Xuất keystream, phát xung valid_o | 1 |

**Tổng thời gian xử lý: 23 chu kỳ xung nhịp cho mỗi block 512-bit.**

Trong trạng thái S_ROUND, 4 instance của chacha20_qr hoạt động **song song** trong cùng một chu kỳ. Các vòng chẵn (round_q[0] = 0) thực hiện column round, các vòng lẻ thực hiện diagonal round theo RFC 8439 §2.1.3:

- **Column round**: QR(0,4,8,12), QR(1,5,9,13), QR(2,6,10,14), QR(3,7,11,15)

- **Diagonal round**: QR(0,5,10,15), QR(1,6,11,12), QR(2,7,8,13), QR(3,4,9,14)

### 3.5.4 Giao tiếp TileLink (tlul_chacha20)

Module tlul_chacha20 cung cấp register map theo Bảng 3.X cho phép CPU cấu hình và điều khiển lõi ChaCha20.

**Bảng 3.X – Register map ChaCha20**

| Offset | Tên | R/W | Mô tả |
|---|---|---|---|
| 0x00–0x1C | KEY[0..7] | R/W | Khóa bí mật 256-bit (8 word) |
| 0x20–0x28 | NONCE[0..2] | R/W | Nonce 96-bit (3 word) |
| 0x2C | COUNTER | R/W | Block counter 32-bit |
| 0x30 | CONTROL | W | Bit[0]=start (tự xóa) |
| 0x34 | STATUS | R | Bit[0]=ready (1 khi rảnh) |
| 0x38 | IRQ_STATUS | W1C | Bit[0]=done IRQ |
| 0x40–0x7C | PLAINTEXT[0..15] | R/W | Bản rõ 512-bit |
| 0x80–0xBC | CIPHERTEXT[0..15] | R | Bản mã 512-bit (chỉ đọc) |

**Luồng hoạt động** của CPU khi sử dụng bộ tăng tốc:

1. Ghi KEY, NONCE, COUNTER vào các thanh ghi tương ứng
2. Ghi dữ liệu cần mã hóa vào PLAINTEXT[0..15]
3. Ghi 1 vào CONTROL[0] để kích hoạt
4. Poll STATUS[0] cho đến khi bằng 1 (lõi hoàn thành)
5. Đọc kết quả mã hóa từ CIPHERTEXT[0..15]

Sau mỗi block hoàn thành, COUNTER tự động tăng thêm 1 theo đúng quy định của RFC 8439 §2.4, sẵn sàng cho block tiếp theo mà không cần CPU can thiệp.

---

## 3.6 Poly1305 MAC Hardware Accelerator

### 3.6.1 Tổng quan

Poly1305 là thuật toán tạo mã xác thực thông điệp (Message Authentication Code – MAC) 128-bit, được định nghĩa trong RFC 8439. Khi kết hợp với ChaCha20, cặp ChaCha20-Poly1305 tạo thành một hệ mã hóa xác thực (AEAD – Authenticated Encryption with Associated Data), trong đó ChaCha20 đảm bảo tính bí mật còn Poly1305 đảm bảo tính toàn vẹn của dữ liệu.

Bộ tăng tốc Poly1305 gồm hai module:

- **poly1305_core**: Lõi tính toán với multiplier tuần tự và mạch rút gọn modular.
- **tlul_poly1305**: Lớp giao tiếp TileLink-UL và register map.

*[Chèn Hình 3.X – Sơ đồ khối Poly1305 Accelerator]*

### 3.6.2 Cơ sở toán học

Poly1305 tính MAC của một thông điệp theo công thức:

```
acc = 0
for each block:
    acc = ((acc + block_with_hibit) × r) mod P
tag = (acc + s) mod 2^128
```

Trong đó:

- **$P = 2^{130} - 5$**: Số nguyên tố đặc biệt cho phép thực hiện phép rút gọn hiệu quả

- **r**: 16 byte đầu của khóa, sau khi áp dụng phép clamp để đảm bảo hiệu quả tính toán
- **s**: 16 byte sau của khóa, được cộng vào kết quả cuối cùng
- **block_with_hibit**: Mỗi block thông điệp được thêm bit 1 ở vị trí byte thứ (len+1) để phân biệt các block có độ dài khác nhau

**Phép clamp r** theo RFC 8439 §2.5 yêu cầu xóa một số bit nhất định của r trước khi sử dụng. Cụ thể, 22 bit được buộc về 0 theo mặt nạ `0x0ffffffc0ffffffc0ffffffc0fffffff`. Phép này được hiện thực bằng mạch tổ hợp đơn giản, chỉ cần nối các bit đúng vị trí mà không tốn tài nguyên logic.

### 3.6.3 Lõi tính toán (poly1305_core)

**FSM điều khiển**:

*[Chèn Hình 3.X – State diagram FSM Poly1305 Core]*

| Trạng thái | Mô tả | Số chu kỳ |
|---|---|---|
| S_IDLE | Chờ lệnh, nạp khóa hoặc kích hoạt xử lý block | 1 |
| S_MULT | Nhân tuần tự sum × r (4 limb 32-bit) | 4 |
| S_REDUCE | Rút gọn tích mod P (2-step Barrett) | 1 |
| S_FINAL | Rút gọn canonical, cộng s | 1 |
| S_VALID | Giữ TAG cho CPU đọc | — |

**Tổng thời gian xử lý: 6 chu kỳ xung nhịp cho mỗi block 16 byte.**

**Nhân tuần tự (Sequential Multiply)**: Phép nhân `sum × r` với sum là số 132-bit và r là số 128-bit được phân rã thành 4 phép nhân con, mỗi phép nhân một limb 32-bit của r trong một chu kỳ:

```
product = sum × r[31:0]  × 2⁰   (cycle 0)
        + sum × r[63:32] × 2³²  (cycle 1)
        + sum × r[95:64] × 2⁶⁴  (cycle 2)
        + sum × r[127:96]× 2⁹⁶  (cycle 3)
```

Kết quả tích lũy vào thanh ghi `product_q` 260-bit.

**Rút gọn modular (Barrett Reduction)**: Sau khi có tích đầy đủ, phép rút gọn mod P được thực hiện qua 2 bước:

```
Bước 1: lo = product[129:0],  hi = product[259:130]
        step1 = lo + 5×hi          (vì 2¹³⁰ ≡ 5 mod P)

Bước 2: lo2 = step1[129:0],  hi2 = step1[132:130]
        step2 = lo2 + 5×hi2        (≤ 2¹³⁰ + 40 < 2¹³¹)
```

Kết quả step2 (131-bit) được lưu vào accumulator, sẵn sàng cho block tiếp theo.

**Finalization**: Sau khi xử lý toàn bộ các block, accumulator có thể vẫn lớn hơn P nhưng tối đa P + 44. Một phép trừ duy nhất `acc - P` là đủ để đưa về dạng canonical. Sau đó cộng s mod $2^{128}$ để có TAG cuối cùng.

### 3.6.4 Giao tiếp TileLink (tlul_poly1305)

**Bảng 3.X – Register map Poly1305**

| Offset | Tên | R/W | Mô tả |
|---|---|---|---|
| 0x00–0x0C | KEY_R[0..3] | R/W | Khóa r 128-bit (4 word) |
| 0x10–0x1C | KEY_S[0..3] | R/W | Khóa s 128-bit (4 word) |
| 0x20–0x2C | MSG[0..3] | R/W | Block thông điệp 128-bit |
| 0x30 | CONTROL | W | [0]=init, [1]=block, [2]=finalize |
| 0x34 | BLOCK_LEN | R/W | Số byte trong block hiện tại (1–16) |
| 0x38 | STATUS | R | [0]=busy, [1]=valid |
| 0x3C | IRQ_STATUS | W1C | [0]=done IRQ |
| 0x40–0x4C | TAG[0..3] | R | MAC tag 128-bit (4 word) |

**Luồng hoạt động** của CPU khi sử dụng bộ tăng tốc:

1. Ghi KEY_R và KEY_S vào các thanh ghi tương ứng
2. Ghi 1 vào CONTROL[0] (init) để khởi tạo, xóa accumulator
3. Với mỗi block thông điệp:
    - Ghi dữ liệu block vào MSG[0..3]
    - Ghi độ dài block vào BLOCK_LEN (1–16 byte)
    - Ghi 1 vào CONTROL[1] (block) để kích hoạt xử lý
    - Poll STATUS[0] (busy) cho đến khi về 0
4. Ghi 1 vào CONTROL[2] (finalize) để tạo TAG
5. Đọc kết quả từ TAG[0..3]